

# Explainable Semantic Textual Similarity via Dissimilar Span Detection

Diego Miguel Lozano<sup>1,†</sup>, Daryna Dementieva<sup>1,2</sup>, Alexander Fraser<sup>1,2</sup>

<sup>1</sup> School of Computation, Information and Technology, Technical University of Munich (TUM)

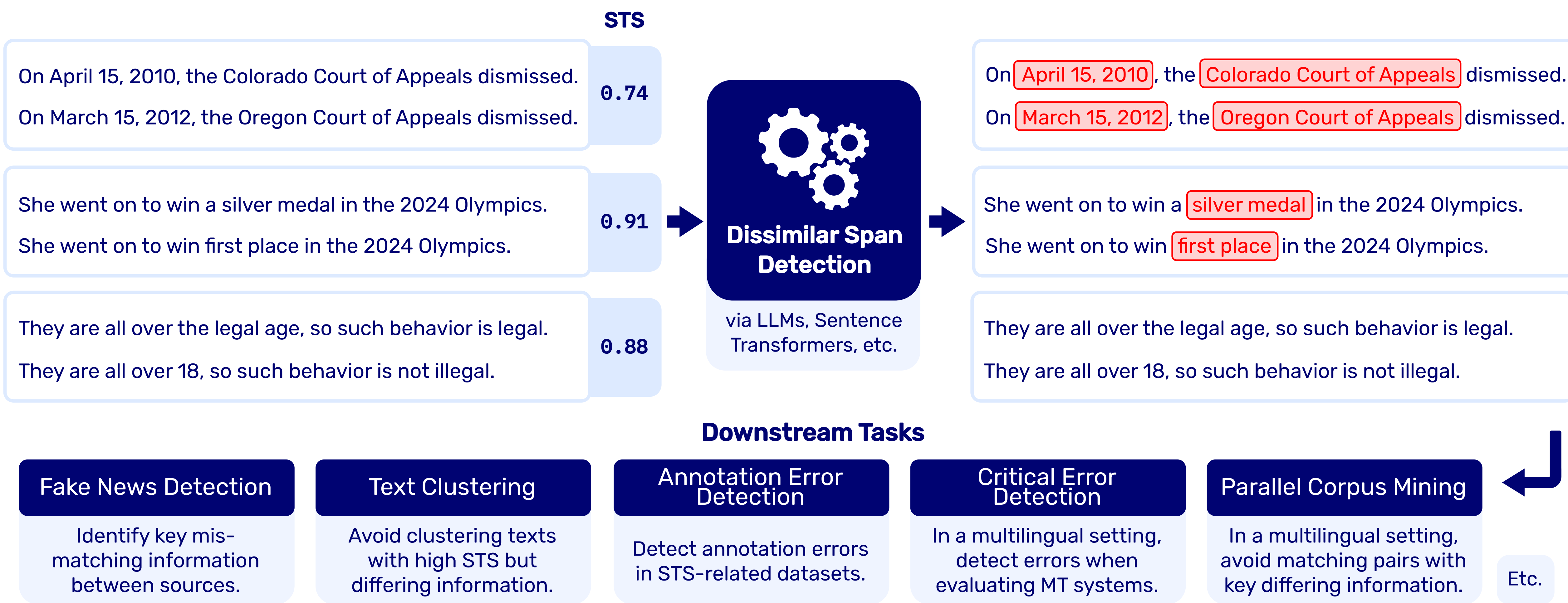
<sup>2</sup> Munich Center for Machine Learning (MCML)

<sup>†</sup> Currently affiliated to ELLIS Alicante



## Cosine similarity is not always enough!

Outputting a single, non-interpretable number can mask fundamental differences between the texts being compared. We introduce the task of **Dissimilar Span Detection (DSD)**: Given two texts, identifying spans pairs with a common semantic function, but differing meanings.



## 1. Span Similarity Dataset (SSD)

New dataset (1,000 samples) crafted specifically for the task of DSD:

| Sentence 1                                                   | Sentence 2                                                      | Span Similarity | Sentence Similarity |
|--------------------------------------------------------------|-----------------------------------------------------------------|-----------------|---------------------|
| It was <b>{{restored}}</b> in the <b>{{1980s}}</b> .         | It was <b>{{destroyed}}</b> in the <b>{{eighties}}</b> .        | 0, 1            | 0                   |
| Thank you for <b>{{wasting my money}}</b> .                  | Thank you for <b>{{misusing my funds}}</b> .                    | 1               | 1                   |
| There are depots at <b>{{Quilpie}}</b> and <b>{{Roma}}</b> . | There are depots at <b>{{Brisbane}}</b> and <b>{{Sydney}}</b> . | 0, 0            | 0                   |

Table 1: Examples of shorter sentences from the SSD. Spans are denoted using double curly braces, and then annotated respectively with a 0, in case the span pair differs in meaning, or 1, if they are equivalent. We also annotate the sentence similarity of the pair as a whole.

Created in a semi-automatic way in two steps: (1) We let an LLM replace the spans, and (2) we manually review them and annotate the similarity. To ensure the correctness of the spans, six annotators manually annotate spans in 100 samples, achieving good inter-annotator and annotator-dataset agreement ( $\kappa \in [0.61, 0.91]$ ).

## 3. Results

| Method                               | Model Size | SSD          |              |              |             | SemEval-2016 |             |
|--------------------------------------|------------|--------------|--------------|--------------|-------------|--------------|-------------|
|                                      |            | F1-Global    | F1-NoDiff    | F1-Diff      | Time        | F1-Diff      | Time        |
| LIME<br>all-mpnet-base-v2            | 109M       | 0.463        | 0.782        | 0.223        | 1981.81     | 0.109        | 199.89      |
| SHAP<br>all-MiniLM-L6-v2             | 22.7M      | 0.366        | 0.434        | 0.131        | 44.47       | 0.306        | 4.52        |
| Embedding<br>all-mpnet-base-v2       | 109M       | 0.469        | 0.529        | 0.423        | 11.28       | 0.352        | 1.13        |
| Embedding<br>text-embedding-004      | Billions   | 0.547        | 0.666        | 0.459        | 611.42      | 0.338        | 34.57       |
| LLM<br>Claude 3.5 Sonnet             | Billions   | <b>0.750</b> | <b>0.895</b> | <b>0.640</b> | 337.80      | 0.389        | 39.22       |
| Token Classification<br>roberta-base | 125M       | 0.690        | 0.838        | 0.574        | <b>1.23</b> | <b>0.441</b> | <b>0.10</b> |

Table 2: Summary of the results. We adopt a 5-fold cross-validation scheme. To account for false positives, we report separate F1 scores for those sentence-pairs that contain no dissimilar spans (*NoDiff*), and those that do (*Diff*). Evaluation times are reported in minutes.

### 3.1 Downstream Task – Paraphrase Detection

| Model             | STS   | STS + DSD    |
|-------------------|-------|--------------|
| all-MiniLM-L6-v2  | 0.720 | <b>0.808</b> |
| all-mpnet-base-v2 | 0.795 | <b>0.868</b> |

Table 3: Comparison of accuracies on the PAWS-Wiki Labeled using uniquely STS or combining STS with DSD.



DSD improves performance on the task with no fine-tuning.

## 2. Experimental Setup

We propose **5 methods** to tackle the task of DSD:

|                                 |                                                                                                                                                                                                                                 |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SHAP-DSD*</b>                | Based on the SHAP framework [1]. We calculate the dissimilarity score as:<br>$\text{DissimilarityScore}(a, b) = 1 - \text{CosineSimilarity}(a, b)$<br>Tokens with dissimilarity above a threshold are considered as dissimilar. |
| <b>LIME-DSD*</b>                | Similar to SHAP-DSD, but with the explanation weights predicted by LIME [2].                                                                                                                                                    |
| <b>LLM-DSD</b>                  | Uses an LLM to detect the dissimilar spans.                                                                                                                                                                                     |
| <b>Token-Classification-DSD</b> | A model is fine-tuned to predict dissimilar spans.                                                                                                                                                                              |
| <b>Embedding-DSD*</b>           | This method constitutes a <b>novel contribution</b> .                                                                                                                                                                           |

We calculate all possible unigram, bigram, ...,  $n$ -gram replacements from the first sentence to the second. For example, for the following 2 sentences:

the bird flies fast over the hill  
the car rides fast over the hill

If we were considering the trigram ['the', 'bird', 'flies'], we would get the replacements:

1 the bird flies fast over the hill  
2 the the bird flies over the hill  
3 the car the bird flies the hill  
4 the car rides the bird flies hill  
5 the car rides fast the bird flies

We then calculate the *similarity gain* for each  $n$ -gram by comparing the resulting cosine similarity to the *base similarity* (i.e., the cosine similarity between the original sentences). Each unigram is assigned the gains coming from each of the  $n$ -grams in which it appears. These are then combined through an *aggregation function*:

$$\text{AggrGain}_{\text{unigram}} = \frac{1}{n} \cdot \sum_{i=1}^n \frac{\text{gains}_i}{i}$$

Unigrams with a gain above a certain threshold are considered as dissimilar.

\* Methods that require a threshold. We find the optimal threshold by evaluating on the validation split of the dataset.

Want to know more?



dmlls.github.io/  
dissimilar-span-detection